BLAST Databases

Running BLAST

1. Requires a BLAST database

2. Requires a BLAST program

- **makeblastdb** creates blast databases.
- **blastdbcmd** queries blast databases.
- **update_blastdb.pl** downloads databases.
- **blastn**, **blastp**, **blastx**, **tblastn**, **tblastx** are blast programs

👽 blastdbcmd - the unsung hero'

😾 a useful tool with an unfortunate name

😾 and unfortunate parameters

😓 and unfortunate documentation

😟 with a quirky and nerdy behavior

🙀 all the above leading to semi obscurity

😿 living under a rock, unknown to most

😻 *other than that it is a very useful tool*

blastdbcmd in "action" 🐾

Get the 16S data

```
update_blastdb.pl --passive --decompress 16S_ribosomal_RNA
```

What is stored in this database?

```
blastdbcmd -db 16S_ribosomal_RNA -entry all | head
```

Prints the content of the database:

```
>NR_118889.1 Amycolatopsis azurea strain NRRL 11412 16S ri
GGTCTNATACCGGATATAACAACTCATGGCATGGTTGGTAGTGGAAAGCTCCGGCGGT
CAGCTTGTTGGTGGGGTAATGGCCTACCAAGGCGACGACGGGTAGCCGGCCTGAGAGG
...
```

Get data by accession number

If you knew the accession number:

```
blastdbcmd -db 16S_ribosomal_RNA -entry NR_026088 | head -
```

Prints:

```
>NR_026088.1 Alloiococcus otitis strain NCFB 2890 16S ribo
GAGTTTGATCCTGGCTCAGGACGAACGCTGGCGGCATGCCTAATACATGCAAGTCGAA
GAAGAGGCGGAGTGGCGAACGGGTGAGTAACACGTGAGGAACTTGCCCATAAGAGGGG
```

It can format outputs

Detailed but hard to read help

```
blastdbcmd -help
```

Output formats:

```
-outfmt <String>
   Output format, where the available format specifiers ar
        %f means sequence in FASTA format
        %s means sequence data (without defline)
        %a means accession
        %i means sequence id
        %t means sequence title
        %l means sequence length
        %h means sequence hash value
        ...
```

A variety of outputs

Get the name and lenght for one entry:

```
blastdbcmd -db 16S_ribosomal_RNA -entry NR_026088
                                  -outfmt "%a %l" | head -3

NR_026088.1 1525
```

You can even select sequence ranges:

```
blastdbcmd -db 16S_ribosomal_RNA -entry NR_026088
                   -outfmt "%a %l %s" -range 1-20 | head -3
```

prints accession number, sequence lenght and start:

```
NR_026088.1 1525 GAGTTTGATCCTGGCTCAGG
```

Get all entries in the database

Iterate over all entries in the database:

```
blastdbcmd -db 16S_ribosomal_RNA
                -entry all  -outfmt "%a %l" | head -5
```

Prints:

```
NR_118889.1 1300
NR_118899.1 1367
NR_074334.1 1492
NR_118873.1 1492
NR_119237.1 1492
```

You can reformat/sort the output

The longest 16S genes in the database

```
blastdbcmd -db 16S_ribosomal_RNA -entry all
        -outfmt "%a %l" | sort -k 2rn,2 | head -1
```

Prints:

```
NG_046384.1 3600
```

What are the most common 16S sequence starts?

```
blastdbcmd -db 16S_ribosomal_RNA -entry all  -outfmt "%s"
          -range 1-10 | sort | uniq -c | sort -rn | head
```

Let's perform a protein alignment

Get the proteins submitted for the ebola project

```
esearch -db protein -query PRJNA257197 |
                        efetch -format fasta > proteins.fa
```

Run a quick statistics on the file:

```
seqkit stat proteins.fa
```

It is a good idea to investigate every step:

```
file          format  type     num_seqs    sum_len  min_len
proteins.fa   FASTA   Protein     2,240  1,367,025      217
```

Create the blast datatbase

Important settings. Set the type of the sequences:

```
-dbtype prot
```

When the FASTA header is in the NCBI format:

```
-parse_seqids
```

The BLAST datbase build command is then:

```
makeblastdb -in proteins.fa -dbtype prot -out ebola
                                              -parse_seqids
```

Investigate your BLAST database

Print the accession number and sequence length:

```
blastdbcmd -db ebola -entry 'all' -outfmt "%a %l"  | head
```

will print:

```
AKC37233.1 2212
AKC37232.1 251
AKC37231.1 288
AKC37230.1 297
AKC37229.1 364
```

Using BLAST

Question: How did the polymerase gene of the ebola virus change between the **1972** and the **2014** outbreaks?

The sequence accession from the 1972 outbreak is `AAD14589` (aka gene named `L`)

Get the sequence (I like to get it in both formats because genbank has more information in it):

```
efetch -db protein -id  AAD14589 -format genbank  >  L.gb
efetch -db protein -id  AAD14589 -format fasta  >  L.fa
```

Run the protein BLAST

This generates too much of output:

```
blastp -db ebola -query L.fa
```

Reformat the results:

```
blastp -db ebola -query L.fa -outfmt "6 qacc sacc  pident"
```

Prints:

```
AAD14589.1      AKC36001        98.734
AAD14589.1      AKC36433        98.734
AAD14589.1      AKC36702        98.734
AAD14589.1      AKC37035        98.734
```

Sort the blast formatted output

Which is the most similar hit (by percent identity):

```
blastp -db ebola -query L.fa -outfmt "6 qacc sacc  pident"
                                    | sort -k3,3rn | head -4
```

We note how none of them are 100% match:

```
AAD14589.1      AIE11805        98.779
AAD14589.1      AIE11814        98.779
AAD14589.1      AIE11823        98.779
AAD14589.1      AIE11832        98.779
```

Extract the top hit from the BLAST database:

```
blastdbcmd -db ebola -entry AIE11805  > AIE11805.fa
```

Investigate the top hits

Blast also supports pairwise alignment:

```
blastp -query L.fa -subject AIE11805.fa
```

Or use our scripts:

```
global-align.sh L.fa AIE11805.fa
```

Evaluate the alignments:

```
...
AAD14589.1      1651 NRKYLARDSSTGSSTNNSDGHIERSQEQTTRDPHDGT
                     |||.|.|:||||||||||||||:||||||||||||||
AIE11805.1      1651 NRKDLTRNSSTGSSTNNSDGHIKRSQEQTTRDPHDGT
...
```

What are blast tasks?

Blast is tuned to different search strategies.

This is an ebola genome:

```
efetch -db nucleotide -id KM233118 --format fasta > KM2331
```

Make it into a blast database:

```
makeblastdb -in KM233118.fa -dbtype nucl -out KM233118 -pa
```

Extract a sequence and align it

Get the first 40 bases of the genome:

```
blastdbcmd -db KM233118 -entry KM233118 -range 1-40
                                                > query.fa
```

Align with blast:

```
blastn -db KM233118 -query query.fa
```

It works!

```
Query  1    AATCATACCTGGTTTGTTTCAGAGCCATATCACCAAGATA  40
            ||||||||||||||||||||||||||||||||||||||||
Sbjct  1    AATCATACCTGGTTTGTTTCAGAGCCATATCACCAAGATA  40
```

BLAST task needs to suit the job

Now make the query shorter:

```
blastdbcmd -db KM233118 -entry KM233118 -range 1-15
                                                > query.fa
```

Now the same alignment:

```
blastn -db KM233118 -query query.fa
```

Prints:

```
***** No hits found *****
```

Select a different BLAST task

The default task is `megablast` change it to `blastn`:

```
blastn -db KM233118 -query query.fa -task blastn
```

Now it works again:

```
Query  1    AATCATACCTGGTTT  15
           ||||||||||||||| 
Sbjct  1    AATCATACCTGGTTT  15
```

Make the query even shorter

```
blastdbcmd -db KM233118 -entry KM233118 -range 1-7
                                                > query.fa
```

Now `blastn` does not work either:

```
blastn -db KM233118 -query query.fa -task blastn
```

But a different task will:

```
blastn -db KM233118 -query query.fa -task blastn-short
```

In general we have to be cautios when BLAST generates no hits. Sometimes the query properties make BLAST filter out the results.

makeblastdb `-parse_seqids` option

Using the "parse_seqids" flag when invoking makeblastdb allows the retrieval of sequences based upon sequence identifiers.

But in that case, each sequence must have a unique identifier, and that identifier must have a specific format

See also section **5.14 Limiting a Search with a List of Identifiers** in the NCBI BLAST+ handbook